

Machine Learning

Assignment 2

Scenario 1

You have trained a Logistic Regression model to detect fraudulent transactions. Your test set contains 10,000 transactions, out of which only 50 are actual frauds. Your model predicts that every transaction is legitimate (non-fraudulent).

Question 1: Confusion Matrix

Ans: The model is predicting every single transaction as legitimate, so it never actually flags anything as fraud. Because of this, all 50 real frauds get missed and they become False Negatives. The remaining 9,950 legitimate ones are classified correctly as True Negatives. There is no True Positive and no False Positive at all.

	Predicted Fraud	Predicted Legitimate
Actual Fraud	TP = 0	FN = 50
Actual Legitimate	FP = 0	TN = 9,950

Question 2: Accuracy, Recall, and Why Accuracy is Flawed

Ans:

Accuracy tells us what proportion of all predictions was correct:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{0 + 9950}{0 + 9950 + 0 + 50} = \frac{9950}{10000} = 99.5\%$$

Recall (also called Sensitivity) tells us how many of the actual positives the model was able to catch:

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{0}{0 + 50} = 0\%$$

Even though the accuracy looks really high at 99.5%, it is actually a very misleading number here. The classes are heavily imbalanced because only 50 out of 10,000 transactions are fraud, which is just 0.5%. So if a model simply labels everything as legitimate without even learning anything, it still gets 99.5% accuracy. In fraud detection, missing a real fraud (False Negative) is way more costly than wrongly flagging a normal transaction, so we should be looking at metrics like Recall, Precision, or F1-score instead of accuracy.

Question 3: SMOTE vs. Cost-Sensitive Learning

Ans:

SMOTE (Synthetic Minority Over-sampling Technique) works by first picking a sample from the minority class and then finding its k nearest neighbours in feature space (usually $k = 5$). After that it randomly picks one of those neighbours and creates a new fake sample somewhere on the straight line between the two points. The math behind it is simple: if x_i is the picked sample and x_{nn} is the neighbour, then the new point is $x_{new} = x_i + \lambda \cdot (x_{nn} - x_i)$, where λ is a random number between 0 and 1. This keeps happening until the minority class has enough samples.

Cost-Sensitive Learning does not touch the data at all. Instead it changes the loss function so that making a mistake on the minority class costs much more. For example, if the model wrongly calls a fraud as legitimate, the penalty could be 100 times bigger than the other way around. So the optimizer naturally tries to avoid those expensive mistakes and moves the decision boundary accordingly.

	SMOTE	Cost-Sensitive Learning
Advantage	It makes the training set richer by adding different synthetic samples, which helps the model learn a wider decision area for the minority class.	It does not create any fake data, so there is no risk of adding noisy or unrealistic samples into the dataset.
Limitation	It can produce noisy or bad samples when minority class points are overlapping with majority class points in feature space.	It needs careful tuning of the cost values. If the costs are set wrong, the model might start over predicting the minority class and give too many false positives.

Scenario 2

Your dataset contains two clusters shaped as concentric rings, an inner ring of Class A surrounded by an outer ring of Class B.

Question 1: Why K-Means Fails on Concentric Rings

Ans: K-Means tries to minimise the Within-Cluster Sum of Squares (WCSS):

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

where μ_k is the centroid of cluster C_k . The problem is that this objective function basically assumes every cluster is a round, compact ball of points grouped around its center. Concentric rings break all of these assumptions. The inner ring sits right inside

the outer ring, so when K-Means calculates the centroids, both centroids end up at almost the same spot near the middle. Then during the assignment step, K-Means gives each point to whichever centroid is closest, but since the two centroids are nearly on top of each other it just cuts the data in half vertically or horizontally instead of separating the rings by their radius. K-Means simply cannot handle this kind of non-convex shape.

Question 2: Single-Linkage vs. Complete-Linkage

Ans: The person arguing for single-linkage is correct for this type of geometry.

Single-linkage measures the distance between two clusters as the shortest distance between any pair of points, one from each cluster:

$$d_{\text{single}}(A, B) = \min_{a \in A, b \in B} \|a - b\|$$

Complete-linkage measures the distance as the longest distance between any pair of points from the two clusters:

$$d_{\text{complete}}(A, B) = \max_{a \in A, b \in B} \|a - b\|$$

Single-linkage works well for concentric rings because points sitting next to each other on the same ring are very close together, so they form a chain. Single-linkage follows this chain and groups the whole ring into one cluster naturally. The two rings stay separate because the gap between them is bigger than the spacing between neighbouring points on the same ring.

Complete-linkage fails here because it looks at the farthest pair of points when deciding whether to merge. Points on opposite sides of a single ring are very far apart (basically the diameter of that ring), so complete-linkage sees that big distance and might actually prefer merging a piece of one ring with a nearby piece of the other ring. This tendency to form tight, round clusters makes complete-linkage unsuitable for long or ring shaped structures.

Scenario 3

You are building a model to predict student exam scores based on features like study hours, attendance rate, and department (a categorical variable). Your preprocessing involves a **StandardScaler** for numerical features and a **TargetEncoder** for the categorical feature. A junior teammate proposes the following order: fit both transformers on the full dataset → split into train/test → train the model.

Question 1: Identifying the Leakage

Ans: This is **Data Leakage**, specially the type known as **Train-Test Contamination** (also called preprocessing leakage).

When you fit the **StandardScaler** on the whole dataset, the mean and standard deviation it learns already include test set values. So the training data gets transformed using numbers that come partly from the test set. The **TargetEncoder** is even more problematic because it replaces each department with the average exam score of students in that department, and those averages are computed from all the rows including test rows. This means the encoded training features already contain information about the test set target values. The model then gets indirect knowledge of what it is supposed to predict during testing, which makes the evaluation scores look better than they really are.

Question 2: Correct Pipeline Order

Ans: The correct way to set up the pipeline is:

Step 1: Split the full dataset into training set and test set (for example 80/20).

Step 2: Fit the **StandardScaler** on just the training set (learn the mean and standard deviation from training data only).

Apply (transform) the fitted scaler to both training set and test set.

Step 3: Fit the **TargetEncoder** on just the training set (learn how each category maps to the target using training labels only).

Apply (transform) the fitted encoder to both training set and test set.

Step 4: Train the model on the transformed training set.

Step 5: Evaluate the model on the transformed test set.

The main rule here is: always fit on training data only, then apply the transform to both training and test data. Using a `sklearn.pipeline.Pipeline` with cross-validation can automatically handle this whole thing and helps prevent leakage.

Question 3: Refuting the “Negligible Effect” Argument

Ans: The teammate’s argument does not hold up and here is why. The **TargetEncoder** replaces each department value with the average exam score of students in that department. Even though the test set is only 20% of the data, those test scores still go into calculating the averages. So when the model trains on those encoded features, it is basically getting a hint about the scores it will need to predict later. This creates a circular problem where training features are partly coming from test labels. Also, if a department only have a few students, even one or two test scores can shift the encoded mean by a lot, causing the model to memorise test specific patterns. The whole point of having a test set is to simulate data the model has never seen before. Any amount of leakage, even if it seems small, breaks this principle and the reported error becomes unreliable. In real scenarios, leakage from target encoding has been shown to increase R-squared scores by noticeable margins, sometimes making a bad model look like an excellent one.

Scenario 4

A deployed model predicting daily ride-sharing demand performed excellently for the first three months. By month six, the Mean Absolute Error (MAE) has tripled, despite the API and servers functioning perfectly.

Question 1: Data Drift vs. Concept Drift

Ans:

Data Drift (also called covariate shift) happens when the distribution of input features changes over time but the actual relationship between those features and the target stays the same. Basically the model was trained on data that looked different from what it is now receiving in production.

Ride-sharing example: Suppose a new subway line opens in the city around month four. This changes how people commute, so the distribution of trip starting locations and rush hour demand shifts a lot. The model still knows the right way to map features to demand, but the feature values it is getting now are outside the range it was trained on.

Concept Drift happens when the relationship between inputs and the target itself changes, even if the feature distributions look the same. The real world mapping from features to demand has shifted.

Ride-sharing example: A competing ride-sharing app launches big discounts in month five. Now even for the same weather, time of day, and location, the actual demand on your platform dropped because riders are switching to the competitor. The features look similar to what the model has seen before, but the target (your platform's demand) now has a completely different relationship with those features.

Question 2: Implementing a KS Test for Feature Drift

Ans:

(a) **What two distributions the test compares:**

The KS test compares the cumulative distribution function (CDF) of each input feature from the reference window (this is the original training data or a recent time period when the model was working well) against the CDF of the same feature from the current production window (for example the most recent week of live data). This comparison is done separately for every numerical feature.

(b) **What the test statistic measures:**

The KS test statistic D gives us the biggest absolute difference between the two CDFs:

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{prod}}(x)|$$

Here F_{ref} is the CDF of the reference data and F_{prod} is the CDF of the production data. A bigger D value means the two distributions are more different from each other. The value goes from 0 (meaning both distributions are the same) to 1 (meaning they do not overlap at all).

(c) **How a significant result ($p < 0.05$) triggers a response:**

If the KS test gives a p -value below 0.05 for any feature being monitored, an automated alert gets sent to the ML engineering team. The pipeline logs which feature drifted along with the D statistic and the p -value to a monitoring dashboard. Depending on how bad the drift is, the team can either flag the model for manual review, automatically start a retraining job using recent labelled data, or temporarily switch to a simpler backup model while the main model gets retrained and checked.

Question 3: Red Teaming and Adversarial Inputs in Tabular Regression

Ans:

Red Teaming method: Before launching the ride-sharing demand model, a red team could try feeding it extreme but still believable input combinations to see how it reacts. For example, they could simulate a scenario where a huge concert ends at 11 PM and it is also raining heavily, or they could try rare combinations like a public holiday falling on a weekday with unusually high temperatures. The idea is to check whether the model gives crazy predictions for these edge cases. If the model outputs a negative demand or a number that is way too high compared to anything seen in history, that gets flagged and fixed before the model goes live.

Adversarial Inputs in tabular regression vs. image domain:

In the image domain, an adversarial input is when someone adds tiny changes to the pixel values of an image that are invisible to the human eye but cause the classifier to predict a completely wrong class. For example a picture of a panda gets classified as a gibbon after adding carefully crafted noise. These attacks use gradient information from the model to figure out exactly which pixels to change.

In tabular regression data, an adversarial input is a data row where one or two numbers have been changed by small amounts to make the model give a very wrong prediction. The key differences are:

- In tabular data every feature has a clear meaning and a valid range (like temperature has to be between -20 and 50 degrees), so any change must stay within those realistic limits.
- Tabular features are a mix of continuous, categorical, and ordinal values, so gradient based attacks designed for continuous pixel values do not directly work here.
- There is no “invisible to the eye” concept like in images, but the changed data point still needs to look like a real data point, otherwise basic input checks would catch it easily.